COMP 7640 Group6 E-Commerce System

1. Name: XIANG Bojie Student ID: 25400444
2. Name: ZHOU Tianchi Student ID: 25409638
3. Name: ZHU Tianxiao Student ID: 25407732
4. Name: ZHU Yanbing Student ID: 25423630
5. Name: ZHANG Xinyue Student ID: 25409883
6. Name: LAM Tak Ming Edwin Student ID: 25462423

1 Database Design

1.1 ER Diagram

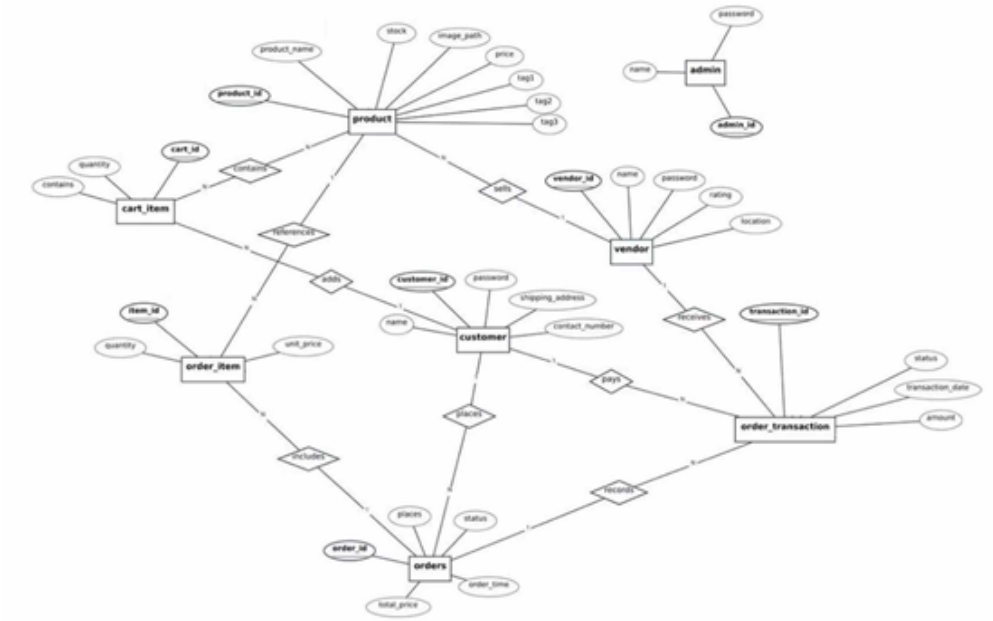

Figure 1: ER Diagram of the E-Commerce System

1.2 1-M Relationship (Unidirectional One-to-Many)

Table 1: ER Diagram Relationship Details

Main Entity (1-side)	Detail Entity (M-side)	Relationship Logic	Foreign Key Location
vendor	product	One vendor can publish multiple products, and one product belongs to only one vendor.	`product.vendor_id` (FK references `vendor.vendor_id`, ON DELETE RESTRICT)
customer	cart_item	One customer can have multiple cart items, and one cart item belongs to only one customer.	`cart_item.customer_id` (FK references `customer.customer_id`, ON DELETE RESTRICT)
customer	orders	One customer can place multiple orders, and one order belongs to only one customer.	`orders.customer_id` (FK references `customer.customer_id`, ON DELETE RESTRICT)
orders	order_item	One order can contain multiple order items, and one order item belongs to only one order.	`order_item.order_id` (FK references `orders.order_id`, ON DELETE RESTRICT)
orders	order_transaction	One order can correspond to multiple payment records (for cross-store orders), and one transaction belongs to only one order.	`order_transaction.order_id` (FK references `orders.order_id`, ON DELETE RESTRICT)
vendor	order_transaction	One vendor can receive multiple payments from different orders, and one transaction belongs to only one vendor.	`order_transaction.vendor_id` (FK references `vendor.vendor_id`, ON DELETE RESTRICT)
product	cart_item	One product can be added to multiple carts, and one cart item belongs to only one product.	`cart_item.product_id` (FK references `product.product_id`, ON DELETE RESTRICT)
product	order_item	One product can be included in multiple orders, and one order item belongs to only one product.	`order_item.product_id` (FK references `product.product_id`, ON DELETE RESTRICT)

1.3 M-M Relationship (Many-to-Many, Implemented Through Junction Tables)

Table 2: Many-to-Many Relationships Implemented via Junction Tables

Entity A	Entity B	Junction Table	Relationship Logic
customer	product	cart_item	A customer can add multiple products to the cart; a product can be added by multiple customers. Linked by composite key (`customer_id` + `product_id`).
orders	vendor	order_transaction	An order can include payments to multiple vendors; a vendor can receive payments from multiple orders. Linked by `order_id` and `vendor_id`.
orders	product	order_item	An order can contain multiple products; a product can appear in multiple orders. Linked by `order_id` and `product_id`.

1.4 Table Designs

We designed 8 core tables based on the ER diagram and actual business requirements. All table structures are shown below:

Table 3: Database Table Designs

Table Name	Core Function	Key Fields
admin	Platform administrator management	admin_id(PK,VARCHAR), name, password
vendor	Merchant information with rating	vendor_id(PK,VARCHAR), name, password, rating, location
customer	User information with delivery details	customer_id(PK,VARCHAR), name, password, contact_number, shipping_address
product	Commodity details with inventory and tags	product_id(PK,AUTO_INCREMENT), vendor_id(FK), product_name, price, stock, tag1, tag2, tag3, image_path
cart_item	Shopping cart records	cart_id(PK,AUTO_INCREMENT), customer_id(FK), product_id(FK), quantity
orders	Order master information	order_id(PK,AUTO_INCREMENT), customer_id(FK), order_time, total_price, status
order_item	Order line item details	item_id(PK,AUTO_INCREMENT), order_id(FK), product_id(FK), quantity, unit_price
order_transaction	Order-vendor payment records	transaction_id(PK, AUTO_INCREMENT), order_id(FK), vendor_id(FK), amount, transaction_date, status

Constraints & Rules:

1. All monetary fields (price, stock, total_price, unit_price, amount) use CHECK (field ≥ 0) to ensure non-negative values.
2. All foreign keys use ON DELETE RESTRICT to prevent accidental data loss.
3. Primary keys of admin, vendor, and customer use custom string identifiers for easier business management.

1.5 Normalization

We applied standard database normalization to eliminate redundancy and prevent insert/update/delete anomalies. Our design fully satisfies 1NF, 2NF, and 3NF.

1.5.1 First Normal Form (1NF)

All fields store atomic single values; no multi-value or composite attributes exist. Product tags are stored in three separate fields (`tag1`, `tag2`, `tag3`). Each field holds a single atomic value, which satisfies the First Normal Form (1NF). This design also meets the project requirement that each product can be assigned a maximum of three tags.

1.5.2 Second Normal Form (2NF)

All tables satisfy 1NF, and non-key attributes fully depend on the **entire primary key** (no partial dependencies). All tables use single-column primary keys, naturally satisfying 2NF.

1.5.3 Third Normal Form (3NF)

All tables satisfy 2NF, with no **transitive dependencies** between non-key attributes. Every non-key field depends only on the primary key. For example, in the `vendor` table, `name`, `rating`, and `location` only depend on `vendor_id`; no non-key field depends on another non-key field.

A special case exists in the `order_transaction` table: the `customer_id` field is theoretically redundant, as customer information can be inferred indirectly through the `order_id` linked to the `orders` table.

However, this is a moderate and reasonable denormalization design to simplify query logic and improve data access efficiency, and it does not form any transitive dependency or violate the constraints of 3NF.

Through three normal forms, our database ensures data consistency, minimizes redundancy, and avoids common data operation exceptions.

2 System Function Modules GUI Implementation

The system provides a complete graphical interface with two independent roles: Administrator (Super Admin) and Customer. All functions are matched with intuitive interactive pages.

2.1 System Login Interface

The system provides a unified login entry named E-Commerce System, supporting role selection for customers and super administrators.

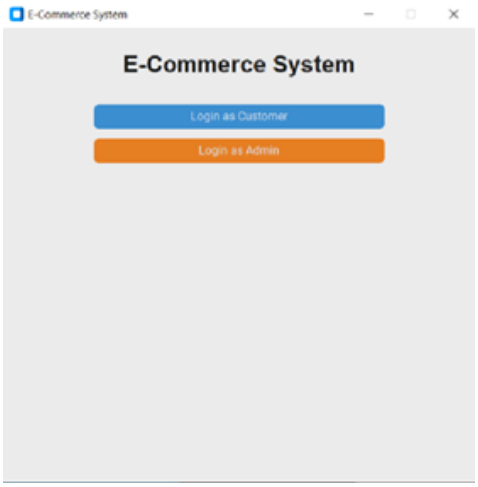

Figure 2: System Login Interface

2.2 Administrator Function Modules (Admin Panel)

The administrator panel includes three core functional tabs, with clear layout and complete management permissions.

2.2.1 Vendor Management

- Left area: Displays the full vendor list
- Right area: Supports adding new vendors and deleting existing vendors

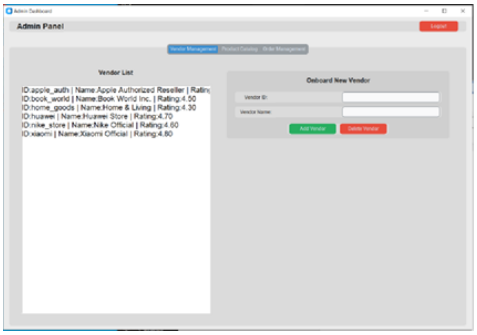

Figure 3: Vender Management

2.2.2 Product Catalog Management

- Top drop-down list: Select different vendors to view their corresponding products
- Bottom form: Input product name, price, stock, and three tags to add products for the selected vendor; supports product deletion

Figure 4: Product Catalog

2.2.3 Order Management

- View all orders submitted by customers, including order details and transaction records

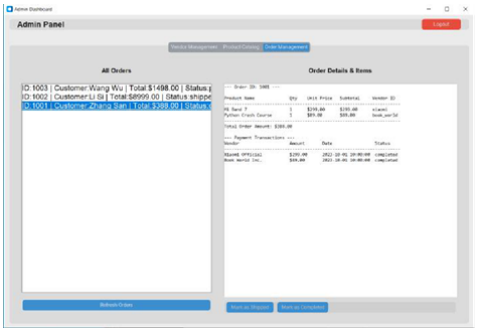

Figure 5: Order Management

- Supports administrators in modifying order status (from pending to shipped to completed) to realize full-process order management.

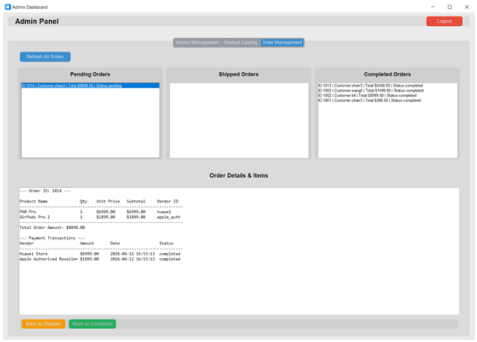

Figure 6: order status

2.3 Customer Function Modules

The customer side provides complete shopping and personal management functions, covering the whole process from product browsing to order completion.

2.3.1 Product Browsing Search

- Display all product information
- Support product search by product name or product tags
- Select target products and add them to the shopping cart

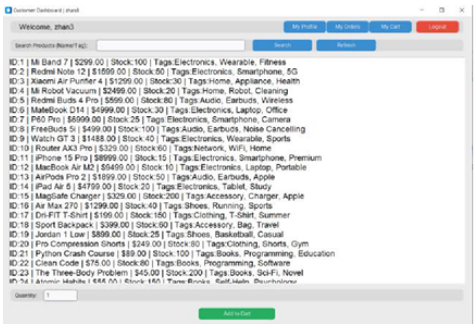

Figure 7: Product Browsing

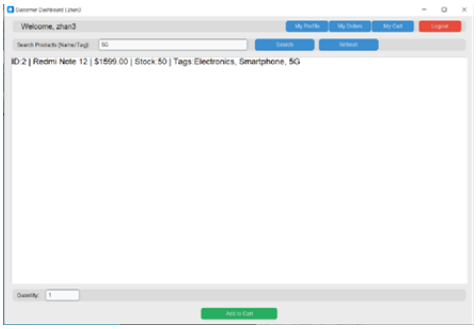

Figure 8: Product Search

2.3.2 Shopping Cart Function

- Modify the quantity of products in the cart
- Delete unwanted products
- Complete checkout to generate orders

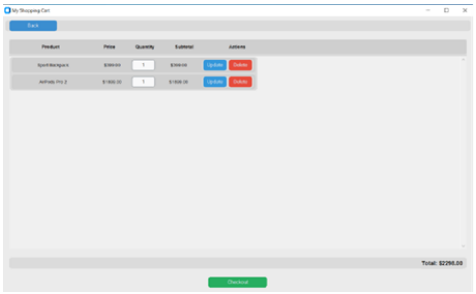

Figure 9: Shopping Cart

2.3.3 Order Management

- View all historical orders in My Orders
- Delete a single product from a pending order
- Remove the entire order record from a pending order

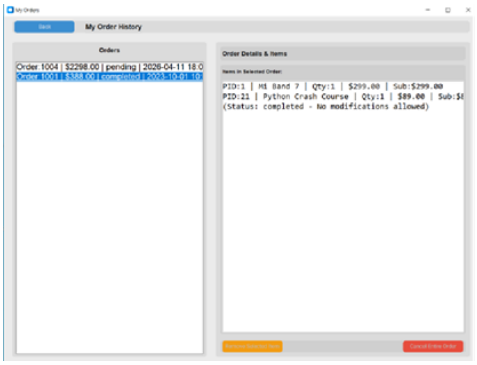

Figure 10: Order Management

2.3.4 Personal Profile Management

- View and edit personal account information: full name, contact number, shipping address

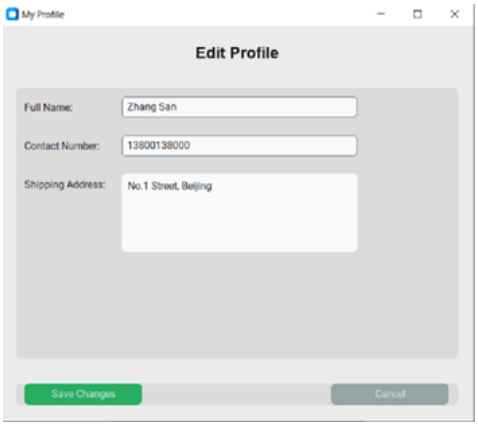

Figure 11: Personal Profile

2.4 Database Interaction Module (database.py)

Encapsulates all database operations to achieve decoupling of GUI and data access:
- User authentication (admin/customer login)
- CRUD operations for vendors, products, carts, orders, and transactions
- Automatic stock update when creating or canceling orders
- Order status management and transaction recording

3 Core Business Processes

3.1 Customer Shopping Process

1. Log in to the customer dashboard → Browse or search products by name/tags → Add products to cart
2. Adjust product quantity or delete items in the cart → Click checkout to generate an order
3. View order details in My Orders → Delete single items or the entire pending order
4. Edit personal profile information in My Profile

3.2 Administrator Management Process

1. Log in to the admin panel → Manage vendor list (add/delete vendors)
2. Select vendors in the product catalog → Manage their products (add/delete products with name, price, stock, tags)
3. View all customer orders and related details in the order management tab → Modify order status to complete the full lifecycle management of orders

4 System Testing and Validation

The system passed full functional testing to ensure stability and correctness:

1. Database Connection Test: Successful connection between Python GUI and MySQL
2. Login Test: Valid authentication for both admin and customer roles
3. CRUD Test: Normal addition, deletion, modification, and query of all business data
4. Business Logic Test: Cart checkout, stock auto-update, order modification, and profile editing work correctly
5. Constraint Test: CHECK constraints prevent negative prices/stock; foreign key constraints protect data integrity
6. Order status modification test: Verifies that administrators can properly update order status.
7. Tag fuzzy search test: Verifies that the system can correctly search products by matching product names or tags.
8. Order modification permission test: Verifies that order modification is only allowed for pending orders.
9. Cross-vendor order transaction record generation test: Verifies that the system can correctly generate transaction records for cross-vendor orders, meeting the project requirement of supporting cross-vendor transactions.

5 Conclusion

This e-commerce database system integrates standardized database design and complete Python GUI application implementation.

The system uses a strict three-layer architecture, complies with database normalization rules (1NF, 2NF, 3NF), and optimizes data security and integrity through constraint configuration and redundant field removal. The desktop GUI provides a friendly interactive experience, supporting core e-commerce functions including role login, vendor management, product management, product search, shopping cart, order processing, and personal profile editing.

The project demonstrates the complete process of database design, optimization, and application integration, achieving high data consistency, security, and system scalability.